

Electronic Companion

Limited-Memory Renewal Contracts: Participation, Quantization, and Exact Design

This companion supplies the detailed proofs and implementation evidence used by the article. It is self-contained together with the article; the broader historical response-compilation program is preserved separately in the repository and is not a hidden premise of the current theorems.

EC.1 Proofs for the Saturated Continuous Frontier

EC.1.1 Proof of Theorem 4.1

Proof. The root promise is the mean of the branch caps, so every branch cap must bind. Hence $y_j = b_j - c_j$ and $0 \leq c_j \leq b_j$. The branch reward $f(c) - h_j(b_j - c)$ is strictly increasing on that interval. Full information therefore uniquely selects $c_j = b_j$; different terminal actions require different symbols.

Within a memory cell all terminal actions coincide and cannot exceed its smallest cap. Monotonicity selects that cap and subtraction from the full-information reward gives (3). Fix any collection of cell minima. Every branch strictly prefers the largest eligible selected minimum, irrespective of its probability and intermediate cost. Reassigning branches in this way keeps each selected minimum in its own nonempty cell and produces contiguous cells without reducing value. Splitting at the final cell gives (4). Expanding the quadratic terms gives the stated prefix-sum implementation. $\square$

EC.1.2 Proof of Theorem 5.1

Proof. The root promise equals the weighted mean of the branch caps. Every branch cap must therefore bind in conditional expectation, so $\mathbb{E}(Y_j + C_j) = b_j$. Replace a randomized terminal decoder, conditional on a symbol, by its mean. It uses the same number of symbols, preserves payments, and weakly increases reward by concavity. Replace a randomized intermediate tier by its branch mean; convexity of h_j weakly lowers its cost without altering the encoder's terminal-symbol probabilities. These replacements use exogenous transitions and the absence of switching costs.

For a terminal mean μ in the convex hull of the levels, the largest expected terminal reward is $F_c(\mu)$. The upper hull of the concave points $(c_i, f(c_i))$ consists of adjacent chords; a lottery over their endpoints attains the hull. The branch objective is $F_c(\mu) - h_j(b_j - \mu)$ on $[c_1, \min(b_j, c_s)]$. It is strictly increasing, so its maximum occurs at μ_j . Feasibility requires $c_1 \leq b_j$ for every branch, equivalently $c_1 \leq b_1$. Subtraction from full information gives the formula. Compactness of symbol probabilities and levels gives attainment, with redundant symbols deleted. Deterministic encodings are admissible randomized encodings, yielding the inequality. The exact-optimum statement follows from the strict-concavity argument in Theorem 8.1, not from the restricted-alphabet formula alone. $\square$

EC.1.3 Proof of Proposition 5.2

Proof. The two deterministic contiguous partitions have losses $7/48$ and $1/8$. For the randomized problem, let the two levels be $u \leq v$. Any choice with $v < 1/4$ can be improved by raising its levels. For $v \geq 1/4$,

feasibility requires $u \leq 1/4$; raising u to $1/4$ weakly increases every relevant concave chord value. Thus take $u = 1/4$.

For v in $[1/4, 1/2]$, $[1/2, 3/4]$, and $[3/4, 1]$, respectively, Theorem 5.1 gives loss

$$\begin{cases} [(1/2 - v)(2 - v) + (3/4 - v)(2 - v)]/3, \\ [(v - 1/2)/8 + (3/4 - v)(2 - v)]/3, \\ [(v - 1/2)/4 + (v - 3/4)/2]/6. \end{cases}$$

The first two expressions strictly decrease on their domains and the third strictly increases. Hence the global minimum is at $v = 3/4$ and equals $1/96$. The first and third branches use their respective endpoint symbols; the middle branch uses each with probability one half. All intermediate tiers are zero. The middle branch's loss is the strict-concavity loss $1/32$, weighted by its probability $1/3$. Three distinct terminal symbols attain full information. $\square$

EC.1.4 Proof of Lemma 6.1

Proof. Attainment follows from the compact level-and-probability formulation in Theorem 5.1. Delete unused levels and coincident duplicates. If all levels lie below b_1 , raising the largest to b_1 improves every branch and leaves only that level relevant. Otherwise delete levels below b_1 except the largest such level, if any, and raise it to b_1 . Every affected cap is bracketed by this level and the next one. Equation (9) shows that raising the lower endpoint weakly lowers loss. This also covers a one-level codebook. The same argument at the other extreme deletes superfluous levels above b_k and lowers the remaining highest endpoint to b_k without increasing loss.

Fix the highest level. Take a nonhighest codeword not equal to a cap. Between the nearest caps and its neighboring codewords, the set of branch caps in each adjacent interpolation interval is fixed. Equation (9) makes the sum of their losses affine in the moving codeword; all other losses are unchanged. Move it to an endpoint that does not increase this affine function. At a cap it becomes anchored. At a collision it can be deleted: no strictly intervening branch cap has been crossed, and the limiting interpolation losses agree. Each move reduces the number of unanchored nonhighest levels or the number of levels. Finitely many moves produce the stated representation. Branches above the fixed highest level are unaffected throughout. $\square$

EC.1.5 Proof of Theorem 6.2

Proof. By Lemma 6.1, a nontrivial optimal codebook consists of an anchored prefix and one final level. The prefix contributes exactly the sum of its edge costs. Splitting at the preceding anchor proves (11) by induction on the number of anchors. If two successive edges share an endpoint cap, its loss is zero, so no term is double counted. Conditional on the final anchor b_j , every possible highest level belongs to one of the intervals in (12). Differentiating that quadratic gives (13); its quadratic coefficient is strictly positive whenever $\ell < k$ because the remaining probabilities and q are positive. Formula (14) therefore optimizes over the entire remaining continuous domain.

Every term in (15) reconstructs a feasible controller with at most m codewords. A coincident final level is deleted. Conversely, the representation lemma maps an optimal controller into one of those terms, including the one-symbol alternative. The two inequalities prove equality, not merely an upper bound or a stationary-point condition. Recover the final level attaining T_j , backtrack the minimizing indices in P , and apply Theorem 5.1 to obtain the encoder probabilities and intermediate tiers.

Prefix sums of π_h , $\pi_h b_h$, $\pi_h b_h^2$, $\pi_h \gamma_h$, $\pi_h \gamma_h b_h$, and $\pi_h \gamma_h b_h^2$ evaluate each edge and each quadratic tail in constant arithmetic work. All T_i require $O(k^2)$ work. The prefix dynamic program requires $O(mk^2)$ work and $O(mk)$ backpointers; edge costs are evaluated on demand rather than stored in a quadratic array. The six moment arrays and all terminal optima require $O(k)$ storage.

For completeness, each moment, edge coefficient, and tail coefficient is a sum of bounded-degree monomials in rational inputs. A conceptual common denominator is a fixed power of the product of input denominators; it has polynomial bit length and is not constructed by the implementation. Each terminal minimizer adds one division of such coefficients. Each prefix cost adds at most k edges; each final candidate adds one tail optimum. Reducing after every operation gives the conservative height bound S . Exact comparisons use cross multiplication on integers of $O(S)$ bits, and ordinary integer arithmetic with greatest-common-divisor reduction costs at most $O(S^3)$ per rational operation. This establishes the stated bound, including comparisons rather than only arithmetic counts. The electronic companion gives an implementation-level concordance. $\square$

EC.1.6 Proof of Proposition 6.3

Proof. The representation lemma fixes the lower level at $1/4$. For an upper level $z \in [1/2, 3/4]$, the loss is

$$\frac{3}{40}(z - 1/2) + \frac{1}{20}(3/4 - z)(2 - z) = \frac{z^2}{20} - \frac{z}{16} + \frac{3}{80}.$$

Its unique minimum is at $z = 5/8$ and equals $23/1280$. On $[1/4, 1/2]$, both higher-cap branches lie above z and the loss strictly decreases up to $1/2$. Levels above $3/4$ can be lowered, as in Lemma 6.1. These observations also exclude the one-level boundary. The cap-only choices with two feasible levels have losses $3/160$ and $3/160$; one level has loss $49/160$. The deterministic partition using levels $1/4, 1/2$ has loss $3/160$, while the other contiguous two-cell partition has loss $21/80$. Theorem 4.1 yields the deterministic optimum.

The middle branch draws terminal level $5/8$ with probability $2/3$ and level $1/4$ with probability $1/3$, with zero intermediate tier. The highest-cap branch uses intermediate tier $1/8$ and terminal tier $5/8$. All branch payments equal their caps in expectation, and the root payment is $17/40$. The resulting weighted losses are $3/320$ and $11/1280$, respectively, summing to $23/1280$. $\square$

EC.1.7 Proof of Theorem 6.4

Proof. Pathwise feasibility gives $Y_j + C_j \leq b_j$ almost surely at each branch. The root promise is $\sum_j \pi_j b_j$ and every branch has positive probability. Hence $Y_j + C_j = b_j$ almost surely on every branch: any positive expected slack would contradict root saturation.

A retained symbol determines a terminal level, after replacing an independent randomized terminal decoder by its mean. This replacement preserves feasibility and weakly improves reward. To see this under pathwise saturation, conditional on a branch and symbol the only permissible additional terminal randomization is fresh and independent of the earlier tier; otherwise a retained shared seed would be an uncharged state. Independence together with $Y_j + C_j = b_j$ almost surely forces that decoder to be degenerate. This also shows why the memory architecture matters.

For a fixed codebook, a symbol with terminal level c can be used on branch j only if $c \leq b_j$; saturation fixes its intermediate tier at $b_j - c$. Its payoff is $f(c) - h_j(b_j - c)$, which is strictly increasing in c . Thus the highest eligible codeword weakly dominates every lottery over eligible symbols, including lotteries with draw-specific intermediate tiers. Selecting it deterministically preserves each branch's payment and cannot reduce payoff. Optimizing codebooks yields the deterministic ordered frontier of Theorem 4.1. The reverse inequality follows because every deterministic feasible controller is an admissible pathwise-randomized controller. Strict-concavity uniqueness gives the final threshold assertion. $\square$

EC.1.8 Proof of Lemma 7.1

Proof. For E , expansion of (10) gives the right-minus-left difference

$$\begin{aligned} \frac{q}{2} \left[(b_d - b_c) \sum_{h=a}^{b-1} \pi_h (b_h - b_a) + (b_b - b_a)(b_d - b_c) \sum_{h=b}^c \pi_h \right. \\ \left. + (b_b - b_a) \sum_{h=c+1}^d \pi_h (b_d - b_h) \right] \geq 0. \end{aligned}$$

Every term is nonnegative, including coincident-index and empty-sum cases. For C , all terms through c cancel and the difference is

$$\sum_{h=c+1}^d \pi_h \{f(b_b) - f(b_a) + h_h(b_h - b_a) - h_h(b_h - b_b)\} \geq 0.$$

Only monotonicity of f and h_h is needed for the second inequality.

Join the functions $\Psi_{i\ell}$ to a continuous function Ψ_i on $[b_i, b_k]$. For $i < j$ and $z \geq b_j$, cancellation gives

$$\begin{aligned} \Psi_i(z) - \Psi_j(z) = \frac{q}{2} \left[\sum_{h=i+1}^j \pi_h (b_h - b_i)(z - b_h) \right. \\ \left. + (b_j - b_i) \sum_{b_j < b_h \leq z} \pi_h (z - b_h) \right]. \end{aligned} \tag{EC.1}$$

The expression is continuous and nondecreasing in z : between caps its slope is nonnegative, and every entering summand is zero at its entry cap. Intermediate-curvature terms cancel. Thus $\Psi_i(x) + \Psi_j(y) \leq \Psi_i(y) + \Psi_j(x)$ for $b_j \leq x \leq y$.

For feasible columns $\ell < t$ with $j \leq \ell$, let x minimize Ψ_j on I_ℓ and y minimize Ψ_i on I_t . Compactness

ensures attainment and interval order gives $x \leq y$, including a common endpoint. Therefore

$$Q(i, \ell) + Q(j, t) \leq \Psi_i(x) + \Psi_j(y) \leq \Psi_i(y) + \Psi_j(x) = Q(i, t) + Q(j, \ell).$$

The singleton final interval is evaluated directly. This proves the inequality after continuous optimization; convexity of each separate minimization would not by itself suffice. $\square$

EC.1.9 Proof of Lemma 7.2

Proof. The two column terms occur once on each side of (18) and cancel. For rows $j < j'$ and columns $i < i'$ with $i' < j$, apply Lemma 7.1 to $i < i' < j < j'$ for E , and $i + 1 < i' + 1 \leq j < j'$ for C . Transposition of the cost arguments gives the required inequality. Unreachable predecessor values are removed, not treated as finite offsets. $\square$

EC.1.10 Proof of Lemma 7.3

Proof. Fix $r < s$ and $c < d$ satisfying the antecedent. There are three possible upper-row configurations.

If both entries are feasible, the strict preference means $A(r, d) < A(r, c)$; equality would select c . If both remain feasible in row s , Monge gives $A(s, d) - A(s, c) \leq A(r, d) - A(r, c) < 0$. If c becomes infeasible but d remains feasible, d wins by its zero first key coordinate. If both become infeasible, $-d < -c$ makes d win. A feasible c with infeasible d is impossible for a suffix.

If c is infeasible and d feasible in row r , the nondecreasing cutoff keeps c infeasible in row s . Either d remains feasible and wins by its first coordinate, or both are padded and $-d < -c$ wins. Finally, if both are infeasible in row r , both remain infeasible in row s and the same padding order applies. These cases are exhaustive. In an entirely infeasible selected row, all keys are $(1, 0, -c)$, uniquely minimized at the largest selected column. Thus rows becoming entirely infeasible after column restriction are included, not excluded by an implicit feasibility assumption. $\square$

EC.1.11 Proof of Lemma 7.4

Proof. If $d > u_r$, either c is feasible and wins by its first key coordinate, or both are padded and $c < d$ wins. The antecedent is false in either case. If the antecedent is true, both upper-row entries are finite. Since $u_s \geq u_r$, both remain finite in row s . A finite tie cannot favor d in row r , and the strict finite inequality propagates by Monge exactly as above. On a wholly padded selected row, $(1, 0, c)$ is uniquely minimized at its smallest column. This covers finite ties, feasible/padded comparisons, and padded/padded comparisons without assigning a numerical infinity. $\square$

EC.1.12 Proof of Lemma 7.5

Proof. An anchored path with s distinct indices ending at j requires $j \geq s$. Conversely, $1, 2, \dots, s-1, j$ gives a finite path whenever $j \geq s \geq 2$. The one-anchor condition fixes $j = 1$. A deterministic partition of j branches into s nonempty ordered cells exists exactly when $j \geq s$. These are necessary and sufficient

characterizations. Discarding endpoint k , which cannot extend to a later endpoint, preserves interval structure. $\square$

EC.1.13 Proof of Theorem 7.6

Proof. The finite-cost, offset, completion, and reachability lemmas verify all hypotheses of classical SMAWK (Aggarwal et al., 1987). Each interface call uses $O(k)$ key comparisons. A request either compares padded indices or performs constantly many moment operations and exact cost evaluations, including repeated requests without caching. The six moment arrays cost $O(k)$ work and space. The terminal pass and at most m program layers cost $O(mk)$. Winner reevaluation and last-anchor scans add $O(mk)$ rather than an uncounted quadratic search.

There are $O(mk)$ predecessors, two active value rows, and $O(k)$ extra search indices. Backtracking one winner for each budget writes at most $\sum_{s=1}^m s = O(m^2) \subseteq O(mk)$ symbols. Reconstruction and output are thus included.

Search introduces no new algebraic depth. A terminal call forms the same six moments, coefficient ratio, clamped level, and substituted quadratic value as Theorem 6.2. A program call adds one edge or cell to a prior path, and a selected path has at most k such costs. Comparisons evaluate a subset, possibly with repetition, of the candidates of quadratic enumeration. Column offsets change no formula; padding uses indices only. The common-denominator argument of that theorem therefore still bounds evaluated rationals and lottery probabilities by $O(S)$ bits. Reduced-fraction arithmetic and cross multiplication cost conservatively $O(S^3)$ per operation. Table EC.1 records the full accounting. $\square$

EC.2 Detailed Renewal Protocol

Consider a stipulated service-batch agreement administered by a renewal desk and executed by a shared dispatch gateway. The desk observes the customer’s current outside alternative and approved continuation. It can perform an advance tier and issue one of a small number of dispatch tickets. The gateway implements the terminal tier named by that ticket; its interface does not retain the renewal record. A separate billing audit can retain the contract without making that record an input to the gateway. Limiting tickets can price gateway-state certification, transmission, or a restricted instruction set. This is a stipulated stylized mechanism, not a claim that a particular industry already uses this architecture.

In the expected-participation version, the offer communicates the advance tier, the terminal ticket levels, and their probabilities. The customer evaluates expected remaining payment before the ticket is drawn and either renews or exits at that review. The protocol commits to the advertised draw. After renewal the desk performs the advance tier, draws the ticket, and sends only that ticket to the gateway. The realized ticket may be disclosed to the customer. “Private” means that the seed is neither a pre-acceptance conditioning variable nor a free future decoder state; it need not mean that realized service is hidden forever. The customer does not receive a new costless exit option after observing an unfavorable draw within this modeled renewal episode.

This version is meaningful only for an ex ante willingness-to-pay or expected-obligation cap and an agreement that permits the stated realization variation. It is not a way to satisfy an ex post enforceable

payment ceiling in expectation. A hard billing limit, a service-safety requirement on every draw, or a review after disclosure instead calls for pathwise feasibility, with the deterministic frontier as its optimum. We make no legal assertion about enforceability. The two feasible sets are explicitly different design alternatives.

Repeated customers who may condition a later renewal on past tickets require that history to enter the later public state, cap, or charged memory. The three-date theorem is not an unqualified repeated-game result. Similarly, a contractible seed disclosed before acceptance changes the conditioning information and hence the participation test. It cannot simply be added as a free source of branch-dependent execution. Section 10 gives a bounded-realization variant for an operator who permits some variation but not the full expected-participation lottery.

EC.3 A Further Nonsaturated Comparison

The earlier nonsaturated two-branch example also remains useful. With branch caps $1/4$ and 1 , root promise $1/2$, and the same quadratic rewards, the full optimum uses terminal tiers $1/4$ and $3/4$ and has value $27/32$. An optimized public-only table uses common terminal tier $1/4$ and intermediate tiers $0, 1/2$, giving $13/32$. The adaptation gain is $7/16$ relative to the optimized restriction. At the full optimum both inherited intermediate tiers are zero, so making that tier observable does not evade this example's exact-memory lower bound. By contrast, the entire restricted-memory frontier above depends on not allowing a branch-specific intermediate tier to serve as an uncharged code.

EC.4 Global Codebook Optimization: Implementation and Verification

This section documents Theorems 6.2 and 6.4. The complete structural and algorithmic proofs appear in this companion; no omitted numerical experiment is being used to supply a proof step.

EC.4.1 Rational implementation concordance

This subsection documents the retained quadratic enumeration engine. The current linear engine uses the same rational primitives but reconstructs only the winning codebook once per budget.

The implementation accepts integers, rational strings, or exact fractions. It rejects binary floating-point inputs, unordered or repeated caps, nonpositive probabilities, probabilities not summing to one, negative intermediate curvature, and terminal parameters outside $r \geq q > 0$. Sorted distinct caps are part of the interface; sorting unorganized observations would first require sorting their associated probabilities and costs together. An arbitrary-order interface would add $O(k \log k)$ rational comparisons.

The six prefix moment arrays evaluate (10) and (12) without summing over an interval at each query. The routine `tail_polynomial` returns the exact coefficients A, B, C of $Az^2 + Bz + C$; `tail` compares all clamped cell minima and saves the attaining final level. The routine `randomized_frontiers` implements (11) using two cost rows and retained predecessor indices, then combines them with terminal costs. A strict improvement stores a reconstructed codebook; ties can keep any incumbent. Duplicate terminal levels are deleted. Budgets greater than the number of branches repeat the zero-loss solution.

The bit bound does not require that the implementation form a common denominator. Every addition, multiplication, and division returns a reduced fraction. Comparisons are exact cross products. Moment arrays contain sums of fixed-degree input products. Edges use products of moment values and cap endpoints. Tail minimizers divide two such coefficients, with a strictly positive divisor on nonterminal cells. Prefix paths add at most k edge costs, and a complete candidate adds one optimized tail. This dependency pattern rules out uncontrolled nested algebraic extensions: all candidates and the reconstructed lottery probabilities are rational. Evaluating final probabilities adds differences and a division of rational codewords and has the same polynomial height bound. The stated conservative S covers both objective values and reconstructed controls, rather than only response coefficients.

For convex nondecreasing intermediate costs that are not quadratic, Lemma 6.1 and the prefix recurrence still apply when terminal reward is quadratic. Each terminal interval instead requires minimizing a strictly convex one-dimensional function. Thus the finite reduction persists with a one-dimensional convex minimization oracle. Exact rational attainment and the particular Turing bound are not transferred to a cost family without an exact representation and such an oracle.

EC.5 Proofs for the Bounded-Overrun Institution

The results in this section use the pre-draw intermediate-action protocol of (27). They do not assert that an arbitrary draw-dependent intermediate-action model reduces to this protocol. All terminal levels and intermediate actions are in $[0, 1]$.

Proposition EC.5.1 (Exact fixed-codebook bounded-overrun rule). *Let f be increasing concave and h_j convex nondecreasing, as in Theorem 4.1. Fix a sorted codebook with lowest level at most b_1 . On branch j , let u be the largest codeword at most b_j . If $u = b_j$, use u deterministically. Otherwise, if there is a next codeword $v > b_j$ and $v - b_j \leq \delta$, an optimal choice is the adjacent-code lottery with mean b_j , with $Y_j = 0$. In every other case an optimal choice is the deterministic codeword u , with $Y_j = b_j - u$.*

Proof. Put $\mu = \mathbb{E}C_j$. The committed intermediate action is $Y_j = b_j - \mu \geq 0$, and every codeword used with positive probability must satisfy $c \leq \mu + \delta$. The greatest expected terminal payoff at a fixed mean is the concave polygonal interpolation $F_c(\mu)$ through the codebook. The function

$$G_j(\mu) = F_c(\mu) - h_j(b_j - \mu)$$

is nondecreasing, and strictly increasing when f is strictly increasing. If $u = b_j$, the deterministic action attains the maximal possible mean. If the top codeword is at most b_j , it similarly attains the maximal possible mean and is optimal.

For the remaining case, $u < b_j < v$. If $v - b_j \leq \delta$, the adjacent lottery with mean b_j satisfies the support bound: its high realization is at most $b_j + \delta$, and its low realization is smaller. It attains both the largest allowable mean and $F_c(b_j)$, and hence the upper bound $G_j(b_j)$.

If $v - b_j > \delta$, no feasible mean can exceed u . Such a mean would require positive mass at a codeword at least v , whereas the realization requirement would imply $v \leq \mu + \delta \leq b_j + \delta$, a contradiction. The

deterministic action at u is feasible, attains $G_j(u)$, and weakly dominates every feasible smaller mean. The argument does not permit a tiny mass on an inadmissible high codeword: an almost-sure constraint excludes any positive mass there. $\square$

Proposition EC.5.2 (Global two-code bounded-overflow example). *For the probabilities, caps, and quadratic rewards in Proposition 6.3, the minimum loss over all codebooks with at most two levels under (27) is (32). In particular, the optimal highest level is continuous in δ on $[0, 1/8]$ and is off-cap for every $\delta > 0$.*

Proof. We first justify restrictions on the two codewords; they are not imposed by the proposed answer. Feasibility requires the lowest codeword to be at most $b_1 = 1/4$. If both codewords are below $1/4$, increasing the highest to $1/4$ improves every branch's deterministic maximal mean without violating a realization bound. Otherwise write the two levels as $u \leq 1/4 \leq z$. Raising u to $1/4$ cannot worsen an optimal branch payoff. For a feasible mean $\mu \geq 1/4$, use the new adjacent mixture with the same mean: its high endpoint z and the overflow condition are unchanged, while the concave quadratic interpolation increases as the low endpoint rises. A feasible mean below $1/4$ is dominated by deterministic service $1/4$, which all branch caps permit. Thus an optimum has lowest codeword $1/4$, including a one-code solution as a repeated level.

A highest level above $b_3 = 3/4$ can be reduced to $3/4$. For any branch mean at most $3/4$, the adjacent interpolation improves as the upper endpoint falls, and the maximum realization decreases. Conversely, if $z < 1/2$, both the second and third branches are at the top codeword and improving z to $1/2$ strictly raises their objective with intermediate service adjusted to keep total payment equal to the cap. The first branch remains at $1/4$. Hence it suffices to optimize $z \in [1/2, 3/4]$.

If $z > 1/2 + \delta$, Proposition EC.5.1 forces the middle branch to use the lower codeword deterministically. Its weighted loss alone is

$$\frac{3}{5}\{f(1/2) - f(1/4) + \frac{1}{2}(1/2 - 1/4)^2\} = \frac{21}{80}.$$

This exceeds the total feasible loss $3/160$ achieved by $(1/4, 1/2)$. Such a codebook is never optimal. For $z \in [1/2, \min\{3/4, 1/2 + \delta\}]$, the middle branch instead uses its mean-preserving adjacent lottery, while the highest branch takes deterministic terminal service z and intermediate service $3/4 - z$. Direct substitution gives

$$\Psi(z) = \frac{z^2}{20} - \frac{z}{16} + \frac{3}{80}, \quad \Psi'(z) = \frac{z}{10} - \frac{1}{16}.$$

The unique unconstrained minimum is $z = 5/8$. Clamping it to the proven feasible interval gives $z_\delta = 1/2 + \min\{\delta, 1/8\}$. The one-code loss is $49/160$, larger than the two-code value throughout. The displayed formula is therefore the global at-most-two-code optimum. It reduces to $3/160$ at $\delta = 0$ and $23/1280$ for $\delta \geq 1/8$. $\square$

The last statement about being off-cap refers to the level itself, not a discrete number of feasible candidates. In particular $5/8$ remains off-cap for every larger δ as well. This example demonstrates a continuously moving operational design under a bounded realization policy. It is not a proof that the general bounded-overflow codebook problem has the same Monge structure as the expected-participation problem.

EC.6 Exact Nonsaturated Allocation and Supporting-Price Certificates

This section makes Proposition 8.4 computational: the targets, encoder, and a certificate are recovered with rational arithmetic for each fixed book. It also supplies a small-instance exact reference solver when allowable books come from a finite catalog. This does not discretize the unrestricted continuous problem or assert pathwise–deterministic equality away from saturation.

For quadratic primitives and fixed $c = (c_1, \dots, c_s)$, define adjacent slopes

$$\sigma_\ell = \frac{f(c_{\ell+1}) - f(c_\ell)}{c_{\ell+1} - c_\ell} = r - \frac{q}{2}(c_\ell + c_{\ell+1}), \quad 1 \leq \ell < s.$$

They are strictly decreasing and positive. The branch response in (25) is affine with these slopes until c_s , and thereafter has derivative $-\gamma_j(t - c_s)$. Let $\mathcal{T}_j = [c_1, b_j]$ and assume $c_1 \leq \min_j b_j$ and $c_1 \leq B \leq \bar{b}$.

Proposition EC.6.1 (Exact allocation with a checkable supporting price). *A fixed-book optimizer and a rational multiplier λ can be obtained in $O(ks + k \log(k + 1))$ exact-rational arithmetic and comparison work. The result obeys*

$$\begin{aligned} \sum_j \pi_j t_j &= B, \quad t_j \in \mathcal{T}_j, \\ \sum_j \pi_j v_{j,c}(t_j) &= \lambda B + \sum_j \pi_j \max_{u \in \mathcal{T}_j} \{v_{j,c}(u) - \lambda u\}. \end{aligned} \tag{EC.2}$$

The right side is an upper bound for every feasible allocation, so equality certifies global fixed-book optimality. This remains valid with repeated caps, zero intermediate curvatures, singleton books, and endpoint promises. Recover terminal means $\min(t_j, c_s)$, adjacent mean-preserving tickets, and intermediate tiers $(t_j - c_s)_+$.

Proof. Initialize every target at c_1 . Increase the targets through successive chord intervals. At interval $[c_\ell, c_{\ell+1}]$, branch j offers capacity $\pi_j [\min(b_j, c_{\ell+1}) - c_\ell]_+$, all at the same marginal value σ_ℓ . Fill these capacities until the required aggregate promise is reached, using any fixed order to divide a tied marginal segment. If the promise is reached in this interval, take $\lambda = \sigma_\ell$. Earlier intervals have larger marginal value, later ones have smaller marginal value, and capped coordinates need no further increase. Thus each chosen target maximizes its branch response minus λ times its target.

If all chord intervals are filled, $t_j = \min(b_j, c_s)$. Fill zero-curvature tails first at price zero. If additional promise is required, only positive-curvature tails remain. Their increments have the form

$$x_j = \min\{b_j - c_s, w/\gamma_j\}, \quad b_j > c_s, \gamma_j > 0,$$

with common water level $w \geq 0$ and $\lambda = -w$. Sort the cap events $w_j = \gamma_j(b_j - c_s)$. Between events the weighted total increment is affine in w , with positive slope $\sum_{j \text{ uncapped}} \pi_j/\gamma_j$. One rational division gives the required water level in the first interval that covers the remaining promise. Tied events are processed together or sequentially with zero intervening width. At the last event all tails are full, so the procedure covers $B = \bar{b}$ as well.

In every case the common multiplier supports every branch response on its feasible interval, proving (EC.2). The chord scan costs $O(ks)$, and the sorted tail sweep costs $O(k \log(k+1))$. Independently checking the certificate enumerates each branch's feasible codeword kinks, its two domain endpoints, and its clamped quadratic stationary point. There are $O(s)$ such candidates per branch, so checking does not change the bound. These operations preserve rationality. The bound counts exact-rational operations, not constant-time arithmetic on arbitrarily large integers. $\square$

Finite-catalog optimization at any promise. Enumerate feasible subsets of an input catalog of size N , with at most m levels. For each, apply Proposition EC.6.1 and subtract the charges of its actually selected levels. The largest certified value is the exact expected-participation catalog optimum at that promise. The work is at most

$$O\left(\sum_{s=1}^{\min(m,N)} \binom{N}{s} [ks + k \log(k+1)]\right).$$

This exponential reference method provides exact interior-promise comparisons; it is not the polynomial saturated recurrence of Theorem 9.1. A maximum over fixed-book concave value functions need not be concave. In particular, this procedure never swaps the outer codebook maximum with the inner supporting-price minimum. With nonnegative charges a redundant level can be deleted, but enumeration need not assume that all selected levels are used.

Independent validation. The implementation's primal chord and water-level sweep is compared with a separately coded dual-event search. That search scans all chord slopes, zero, and all negative tail-cap slopes, interpolating the aggregate target only between consecutive events. Both implementations are then checked by the finite candidate certificate in (EC.2). This checks the continuous branch maxima exactly, not only values on a target grid. At saturation, exhaustive charged-book allocation is also compared with the independently structured catalog path recurrence. Pairwise policy-transport tests exercise contractions and partial lifts under expected, pathwise, and deterministic timing, including intermediate tiers conditioned on drawn symbols. The original theorem proofs, not these finite tests, establish the all-input guarantees.

EC.6.1 Executed allocation checks

The executed exact suite reports 1715 primal/dual-event equalities and 6955 supporting branch maxima. It includes repeated caps, zero curvatures, singleton books, endpoint promises, and nonsaturated charged-book selection. Full rational values and checks are in the allocation validation record.

EC.7 Current Computational Methods and Their Scope

EC.7.1 Exact search verification

The generic completion suite enumerates every nondecreasing cutoff sequence in dimensions one through four. Both shrinking feasible suffixes and expanding feasible prefixes are tested. Finite arrays have the form

$r^2 + a_c - trc$, with $t \in \{0, 1\}$ and either zero or alternating signed column offsets. The zero-slope cases deliberately create ties. Every strict preference implication is checked, and both search implementations are compared with direct minima on every nonempty row and column restriction. Completely infeasible selected rows are retained and counted. Two negative controls reject uniform infinite suffix padding with left ties and arbitrary two-sided feasible intervals. These are finite tests of the proof’s cases, not substitutes for the all-size lemmas.

The economic suite combines the deterministic benchmark family with heterogeneous random rational instances generated from seed 370924. It compares all budgets against the quadratic engine and checks direct branch payoffs, saturation, codebook size, and randomized controller replay. The bounded-overrun reference enumerates all pairs of levels, rather than assume the adjacent-code conclusion. A separate grid permits nonanchored lower codewords in the solved risk example. Continuous optimality rests on the analytic propositions.

EC.7.2 Independent matrix-search substitution

The offline `ConcaveMinima` function of Eppstein (2005) returns column minima. Transposing its interface produces the row-minimum oracle required here. The executable function is excerpted without algorithmic changes from the pinned PADS source; its MIT notice and original blob identity are in `pads_excerpt.py`. The experiment replaces only matrix search. Moment construction, contractual cost evaluation, and reconstruction remain the same. This isolates implementation dependence in the search layer and is not presented as an independent proof of the economic model. The predecessor exact suites run unchanged in temporary directories, so their historical result files are never overwritten.

EC.7.3 External finite-network solver

For deterministic participation, a network node represents a layer and an endpoint; an arc appends a nonempty contiguous cell at its direct branchwise loss. For expected participation, a node represents a count and last cap-anchored codeword; an arc appends another anchor at its chord loss. Sink arcs add the continuously minimized terminal cost. A direct source-to-sink arc represents the one-code solution, and the network permits all budgets up to the requested bound. Every cost is assembled from direct rational sums rather than the moment oracle. Continuous terminal minimization is performed interval by interval with its analytically clamped quadratic minimizer.

The node-arc incidence matrix and unit source/sink demand give a linear program. Compiled HiGHS dual simplex is called through SciPy. The floating solution must be within 10^{-7} of an integral path. The selected path is then checked exactly: its fraction-valued cost equals the sink distance from an independent rational shortest-path pass, and every reached arc satisfies the potential inequality. Direct controller replay and comparison with the exact frontier complete the audit. Thus the recorded reduced-network objective has an exact certificate even though the external optimizer uses floating point. The certificate does not re-prove that this finite network represents the continuous contract problem.

EC.7.4 Runtime and memory accounting

Each timed case runs in a new process and computes budgets one through eight. Input construction has a separate timer. Frontier time includes moment setup, every search and reevaluation, scans, and all winning-codebook reconstructions. It excludes direct validation and serialization, which occur after the measurements. The 120-second limit applies to the entire worker process; an exceeded limit is not a completed solver time. Hardware and interpreter versions are recorded with source hashes.

Peak resident memory is Linux process high-water memory in kibibytes, captured before validation; it includes interpreter, input, arrays, and outputs. Separate tracing begins before input construction and records Python-tracked current and peak bytes. It does not measure all native allocations. Traced runtimes are not compared with untraced times. Table EC.4 reports the traced cases.

Separate cProfile runs at 512 branches assign exclusive time to rational arithmetic (including gcd), moment/economic-oracle work, matrix-search and key bookkeeping, and the combined outer frontier scan/reconstruction routine. Inclusive function times are retained in JSON but not added across categories. The outer category cannot distinguish scanning from reconstruction and is labeled accordingly. Table EC.5 reports these exclusive measurements. Single-run timings describe this implementation and runner; they are not a statistical claim of hardware-independent superiority.

EC.7.5 Pinned independent search identity

The independently authored search excerpt is from David Eppstein’s PADS mirror, commit

a6f82ff6f2d60d52a02f7b816ae7b1dfc431b5d9.

The original search blob is 0a94e9e685130f7f79262c52761414c33226214d; the license blob is af71e1c36786aef6226d34e3999787051343ee9a. The distributed excerpt retains the MIT copyright and permission notice. It is an excerpt with an unchanged executable offline function, not a byte-identical copy of the entire upstream file. These identities distinguish the external search layer from the shared contractual oracle.

EC.8 Preserved Continuous-Frontier Measurements

The computational study has three distinct targets: correctness of the implemented interfaces, practical costs of alternative implementations, and evaluation of a finite reduced model by an external solver. It does not calibrate demand or estimate customers’ willingness to accept realization variation. The deployment mechanism in Section 3 and its prices are stipulated design inputs.

Exact checks. The preceding tests enumerate monotone cutoffs in both completion directions, including wholly padded selected rows, finite ties, signed column offsets, and every nonempty row and column submatrix through dimension four. They compare direct minima with both the retained implementation and the independently authored offline search in Eppstein (2005). Separate heterogeneous rational instances compare all budgets against the quadratic and divide-and-conquer engines and replay every returned controller

directly. Bounded-overflow tests use an independent all-pairs lottery calculation and include nonanchored codebook grid checks. The proofs, rather than these finite tests, establish the continuous and all-size statements.

The executed suite checks 784 completed matrices and 282,832 selected-submatrix row minima, including 98,728 wholly padded selected rows. It records 1,890 exact frontier equalities and direct controller replays, 190 bounded-overflow all-pairs equalities, and 33 evaluations of the solved tolerance frontier. The unchanged R33–R36 suites also pass in isolated directories. All 24 independently assembled HiGHS cases yield an exactly certified reduced-network optimum and a matching controller.

Scale and memory. The same-input study extends the inherited family to 16,384 branches. For branch index $j = 1, \dots, k$, it uses $b_j = j/(k + 1)$, $\pi_j = 2j/[k(k + 1)]$, $\gamma_j = (j \bmod 5)/3$, and $r = 2, q = 1$, computing budgets one through eight. Each engine and institution runs in a fresh process with an explicit 120-second process limit. A limit observation is reported as such, never as a measured frontier time. Table EC.2 reports unprofiled time, economic requests, and peak resident memory. The independent PADS comparison substitutes search alone; economic moments and reconstruction remain shared and are not mislabeled independent algorithms.

At the largest completed common size for expected participation, 16,384 branches, the retained SMAWK implementation takes 26.601 seconds against 37.436 for divide-and-conquer, a time ratio of 0.711. At the largest completed common size for pathwise participation, 16,384 branches, the retained SMAWK implementation takes 44.132 seconds against 52.054 for divide-and-conquer, a time ratio of 0.848. Across the 6 completed same-size comparisons, SMAWK is slower in 2. These are observations on one runner and one input family, not a uniform speedup claim. No configured study case exceeded the stated process limit.

Python reduced-fraction arithmetic makes exact equality and coefficient growth auditable, but it is not a performance model for an optimized native implementation. Independent heap-traced runs measure Python-tracked allocations; resident peaks also include the interpreter and imported code. Separately profiled runs decompose exclusive time into rational arithmetic, economic-oracle work, matrix-search/key bookkeeping, and the combined frontier-scan/reconstruction routine. The last category is deliberately not presented as a separately measured reconstruction cost. All preserved measurements and their definitions are reported in the article and its appendices.

External finite-model comparison. We also assemble unit-flow networks directly from branch sums, without the moment oracle, and solve them with compiled HiGHS dual simplex through SciPy (Huangfu and Hall, 2018; Virtanen et al., 2020). The expected-participation network uses the proved continuous terminal minimizations, not a discretization of the highest level. HiGHS operates in floating point. We extract its integral path, reprice it in exact fractions, verify the controller, and certify its reduced-network optimum by exact shortest-path potentials. Table EC.3 separates graph construction, solver time, and exact certification plus replay. This is an external solver comparison for the finite reduction, not a claim that the solver independently proves cap anchoring or that native SMAWK has been benchmarked.

The earlier experiments, including unfavorable small-instance search constants and shared-cache results,

remain unchanged in the preservation archive. The new evidence addresses the remaining implementation questions rather than accumulate synthetic examples as a substitute for operational data. In particular, finite-precision wall time, rational-operation complexity, and contract value are reported as different quantities.

Table EC.1: Accounting for Theorem 7.6; ordered rational input, $1 \leq m \leq k$.

Cost component	Included work or storage
Input validation and six prefix moments	$O(k)$ rational operations; $O(k)$ stored values.
Sorting	Input interface requires ordered distinct caps; otherwise add $O(k \log k)$ exact comparisons and reorder associated primitives.
Continuous terminal minimization	Included in $O(k)$ oracle requests; each interval minimizes its quadratic exactly, including singleton and endpoint cases.
Key comparisons and repeated requests	Included in $O(mk)$ work; a repeated finite request is charged again. No quadratic entry cache is assumed.
Padding comparisons	Included index comparisons; undefined economic costs are never evaluated.
Previous-layer values and backpointers	$O(mk)$ stored values and indices, plus $O(k)$ working search space.
Reconstruction and output	One winning codebook per budget; $O(m^2) \leq O(mk)$ symbols written. No reconstruction at every candidate endpoint.
Rational heights and integer operations	$O(S)$ bits per evaluated rational, $S = O(k[L + \log(k+2)])$; conservative $O(mkS^3)$ bit operations, including reduction and comparison.
Physical bit storage	$O((mk + k)[S + \log(k+2)])$ bits for the stated arrays and indices; not a claim of $O(mk)$ bits.
Inherited public graph and response compiler	Not invoked by the renewal algorithm; their read-only data, compilation, and query costs enter the separate implementation menu when used.
External solver preprocessing	Not part of the frontier theorem. Finite-network construction, floating solver work, and exact certification are measured separately.

Table EC.2: Same-input unprofiled frontier measurements; budgets one through eight.

Institution	Branches	Search	Seconds	Requests	Resident MiB
Expected	256	Divide-conquer	0.3136	12,177	22.7
Expected	256	SMAWK	0.3452	15,376	22.6
Expected	256	PADS	0.3387	14,754	22.7
Pathwise	256	Divide-conquer	0.4071	13,524	22.8
Pathwise	256	SMAWK	0.5904	19,386	22.7
Pathwise	256	PADS	0.5525	18,127	22.6
Expected	4,096	Divide-conquer	7.9101	296,018	36.0
Expected	4,096	SMAWK	6.4134	259,034	33.2
Expected	4,096	PADS	6.3043	247,709	33.4
Pathwise	4,096	Divide-conquer	10.9931	335,422	36.3
Pathwise	4,096	SMAWK	10.7799	329,278	32.4
Pathwise	4,096	PADS	10.1865	307,277	32.8
Expected	16,384	Divide-conquer	37.4357	1,381,513	69.0
Expected	16,384	SMAWK	26.6007	1,039,837	69.1
Expected	16,384	PADS	26.1597	994,145	70.6
Pathwise	16,384	Divide-conquer	52.0540	1,572,352	65.4
Pathwise	16,384	SMAWK	44.1321	1,322,658	66.1
Pathwise	16,384	PADS	41.4011	1,234,124	67.4

Requests count finite economic-oracle evaluations, including reevaluations. Resident memory is the process peak before validation and includes its baseline. A limit denotes the 120-second whole-process limit, not a measured frontier time.

Table EC.3: External compiled solver on independently assembled reduced networks; eight-symbol budget.

Institution	Branches	Arcs	Construction	Solver	Certification
Expected	8	92	0.0041	0.0022	0.0055
Pathwise	8	101	0.0023	0.0022	0.0051
Expected	16	488	0.0300	0.0044	0.0165
Pathwise	16	585	0.0146	0.0047	0.0204
Expected	32	2,240	0.2074	0.0139	0.0453
Pathwise	32	2,897	0.1048	0.0164	0.0620
Expected	64	9,584	1.5880	0.0559	0.1337
Pathwise	64	12,897	0.7787	0.0675	0.1701

All times are seconds. Certification includes exact repricing, rational shortest-path potentials, controller replay, and comparison with the exact frontier. The machine-readable record also includes budgets two and four. Solver time excludes network construction and certification.

Table EC.4: Separate memory runs at 1,024 branches and budgets one through eight; mebibytes.

Institution	Search	Current heap	Peak heap	Resident peak
Expected	Divide-conquer	2.60	3.92	33.4
Expected	SMAWK	0.59	2.62	29.2
Expected	PADS	0.65	2.68	29.3
Pathwise	Divide-conquer	1.96	2.88	29.7
Pathwise	SMAWK	0.53	2.50	29.3
Pathwise	PADS	0.59	2.58	29.3

Tracing starts before input creation and stops before validation. Heap fields are Python-tracked allocations; resident peaks include the process baseline and native allocations. Traced runtimes are not used as the unprofiled comparison.

Table EC.5: Exclusive-time decomposition in separate 512-branch profile runs; percentages.

Institution	Search	Arithmetic	Oracle	Search	Outer	Other
Expected	Divide-conquer	69.5	11.8	1.6	0.2	16.9
Expected	SMAWK	67.5	10.6	5.2	0.1	16.5
Expected	PADS	67.0	10.6	4.6	0.2	17.7
Pathwise	Divide-conquer	72.0	3.6	8.0	0.1	16.3
Pathwise	SMAWK	70.5	3.5	9.8	0.0	16.2
Pathwise	PADS	70.0	3.5	9.5	0.0	17.0

Arithmetic includes reduced-fraction operations and gcd. Oracle and search columns count their own exclusive work, not child arithmetic again. Outer combines frontier scans and reconstruction. Percentages use exclusive profile time and are not percentages of unprofiled wall time; rounding can prevent an exact sum of 100.

EC.9 Additional Catalog Measurements

Table EC.6: Executed synthetic catalog designs; eight-symbol budget.

Family	k	N	δ	Used	Total cost	Seconds
Uniform	32	16	0	8	0.102528	0.0454
Uniform	32	16	1/8	8	0.006468	0.0498
Uniform	32	16	1	8	0.004531	0.0594
Uniform	128	32	0	8	0.101684	0.5574
Uniform	128	32	1/8	8	0.006890	0.6104
Uniform	128	32	1	8	0.002991	0.6877
Clustered	32	16	0	3	0.054181	0.0487
Clustered	32	16	1/8	5	0.003457	0.0556
Clustered	32	16	1	5	0.002931	0.0605
Clustered	128	32	0	4	0.025774	0.6249
Clustered	128	32	1/8	5	0.001207	0.6797
Clustered	128	32	1	5	0.001067	0.7608
Concentrated	32	16	0	8	0.043783	0.0449
Concentrated	32	16	1/8	7	0.002896	0.0494
Concentrated	32	16	1	6	0.001769	0.0558

Family	k	N	δ	Used	Total cost	Seconds
Concentrated	128	32	0	8	0.027476	0.5622
Concentrated	128	32	1/8	6	0.000796	0.6042
Concentrated	128	32	1	6	0.000529	0.6928

Total cost includes actual selected-level charges. Values are displayed in decimal only for readability; every stored objective and replay uses exact fractions. The included timing scope differs from the preserved continuous-frontier measurements.

References

- Aggarwal A, Klawe MM, Moran S, Shor P, Wilber R (1987) Geometric applications of a matrix-searching algorithm. *Algorithmica* 2:195–208. doi:10.1007/BF01840359.
- Chan TM (2021) (Near-)linear-time randomized algorithms for row minima in Monge partial matrices and related problems. *Proceedings of the 2021 ACM–SIAM Symposium on Discrete Algorithms*, 1465–1482. doi:10.1137/1.9781611976465.88.
- Croci M, Fasi M, Higham NJ, Mary T, Mikaitis M (2022) Stochastic rounding: Implementation, error analysis and applications. *Royal Society Open Science* 9:211631. doi:10.1098/rsos.211631.
- Eppstein D (2005) PADS: Python Algorithms and Data Structures. *SMAWK.py*, offline matrix-search implementation, originally written 2002 and revised 2005. University of California, Irvine. MIT-licensed distribution; source version and excerpt are identified in the code supplement.
- Fu M (2024) A tutorial on quantized feedback control. *IEEE/CAA Journal of Automatica Sinica* 11(1):5–17. doi:10.1109/JAS.2023.123972.
- Grønlund A, Larsen KG, Mathiasen A, Nielsen JS, Schneider S, Song M (2017) Fast exact k -means, k -medians and Bregman divergence clustering in 1D. Preprint, arXiv:1701.07204, version 4.
- Huangfu Q, Hall JAJ (2018) Parallelizing the dual revised simplex method. *Mathematical Programming Computation* 10(1):119–142. doi:10.1007/s12532-017-0130-5.
- Kaplan H, Mozes S, Nussbaum Y, Sharir M (2017) Submatrix maximum queries in Monge matrices and partial Monge matrices, and their applications. *ACM Transactions on Algorithms* 13(2), Article 26. doi:10.1145/3039873.
- Klawe MM, Kleitman DJ (1990) An almost linear time algorithm for generalized matrix searching. *SIAM Journal on Discrete Mathematics* 3(1):81–97. doi:10.1137/0403009.
- Pagès G, Wilbertz B (2012) Intrinsic stationarity for vector quantization: Foundation of dual quantization. Author manuscript, arXiv:1010.4642v2, March 27. Original preprint 2010.
- Spear SE, Srivastava S (1987) On repeated moral hazard with discounting. *Review of Economic Studies* 54(4):599–617. doi:10.2307/2297484.
- Thomas J, Worrall T (1988) Self-enforcing wage contracts. *Review of Economic Studies* 55(4):541–554. doi:10.2307/2297404.
- Virtanen P, Gommers R, Oliphant TE, et al. (2020) SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272. doi:10.1038/s41592-019-0686-2.
- Wu X (1991) Optimal quantization by matrix searching. *Journal of Algorithms* 12(4):663–673. doi:10.1016/0196-6774(91)90039-2.

Yao FF (1980) Efficient dynamic programming using quadrangle inequalities. *Proceedings of the Twelfth Annual ACM Symposium on Theory of Computing*, 429–435. doi:10.1145/800141.804691.

Zhang H (2012) Solving an infinite horizon adverse selection model through finite policy graphs. *Operations Research* 60(4):850–864. doi:10.1287/opre.1120.1056.